

Teach Module 31: Kafka Integration with Spring Boot

Source document used only to identify the module topic: docs/Java Software Engineer bootcamp.docx .

Teaching note: This README teaches the topic independently and does not reuse the course material content.

Module Focus

Module 31 focuses on connecting Spring Boot applications to Apache Kafka.

The main ideas are:

- Publishing Kafka messages from Spring Boot using `KafkaTemplate`
- Consuming Kafka messages using `@KafkaListener`
- Sending structured JSON events
- Understanding consumer groups and offsets
- Handling failed messages with retries and dead-letter topics
- Testing Kafka behavior with embedded Kafka

Core Explanation

Kafka is used when services need to communicate asynchronously. Instead of one service directly calling every other service, a service publishes an event to Kafka and other services react to it.

```
Order Service ---> Kafka topic: orders.created ---> Inventory Service
                                     |
                                     ---> Email Service
                                     |
                                     ---> Analytics Service
```

The producer does not need to know who consumes the event. That makes systems easier to scale and less tightly coupled.

Kafka acts like a durable event log. Messages are written to topics, consumers read those messages, and Kafka tracks how far each consumer group has read.

Key Kafka Terms

A `topic` is a named stream of messages.

Examples:

```
orders.created
payments.completed
users.registered
```

A `producer` sends messages to a topic.

A `consumer` reads messages from a topic.

A `consumer group` is a group of consumers that share the work. If three consumers are in the same group, Kafka divides the topic partitions among them so each message is processed once by that group.

An `offset` is Kafka's bookmark. It records how far a consumer has read in a topic.

Spring Boot Kafka Producer

In Spring Boot, the common way to publish Kafka messages is with `KafkaTemplate`.

Example event:

```
public record OrderCreatedEvent(  
    Long orderId,  
    Long customerId,  
    BigDecimal total  
) {}
```

Producer service:

```
@Service  
public class OrderEventProducer {  
  
    private final KafkaTemplate<String, OrderCreatedEvent> kafkaTemplate;  
  
    public OrderEventProducer(KafkaTemplate<String, OrderCreatedEvent> kafkaTemplate) {  
        this.kafkaTemplate = kafkaTemplate;  
    }  
  
    public void publishOrderCreated(OrderCreatedEvent event) {  
        kafkaTemplate.send("orders.created", event.orderId().toString(), event);  
    }  
}
```

The first argument is the topic.

The second argument is the key.

The third argument is the message payload.

A good Kafka key helps Kafka keep related events in order. For order events, `orderId` is usually a sensible key.

Producer Configuration

Spring Boot can configure Kafka through `application.yml`.

```
spring:  
  kafka:  
    bootstrap-servers: localhost:9092  
    producer:  
      key-serializer: org.apache.kafka.common.serialization.StringSerializer  
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
```

The key serializer converts the key into bytes.

The value serializer converts the Java object into bytes.

Kafka stores bytes, not Java objects, so serialization is required.

Spring Boot Kafka Consumer

To consume messages, Spring Kafka commonly uses `@KafkaListener` .

```
@Service
public class OrderEventConsumer {

    @KafkaListener(
        topics = "orders.created",
        groupId = "inventory-service"
    )
    public void handleOrderCreated(OrderCreatedEvent event) {
        System.out.println("Received order: " + event.orderId());
    }
}
```

This method runs whenever a message arrives on `orders.created` .

The `groupId` matters. If `inventory-service` and `email-service` use different group IDs, both services receive the event. If multiple instances of `inventory-service` use the same group ID, Kafka load-balances messages among them.

Consumer Configuration

```
spring:
  kafka:
    bootstrap-servers: localhost:9092
    consumer:
      group-id: inventory-service
      auto-offset-reset: earliest
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.JsonDeserializer
      properties:
        spring.json.trusted.packages: "com.example.orders"
```

Important setting:

```
auto-offset-reset: earliest
```

This tells Kafka what to do when the consumer group has no saved offset yet. `earliest` means start from the beginning. `latest` means only read new messages from now on.

JSON Serialization and Deserialization

When sending Java objects through Kafka, JSON is common because it is readable and language-friendly.

Producer:

```
KafkaTemplate<String, OrderCreatedEvent>
```

Consumer:

```
public void handleOrderCreated(OrderCreatedEvent event)
```

Spring uses Jackson behind the scenes to convert between JSON and Java objects.

Example Kafka message value:

```
{
  "orderId": 101,
  "customerId": 55,
  "total": 249.99
}
```

One practical rule: event classes should be stable. If other services consume your event, changing field names or removing fields can break them.

Error Handling and Dead-Letter Topics

What happens if a consumer fails?

Example:

```
@KafkaListener(topics = "orders.created", groupId = "inventory-service")
public void handleOrderCreated(OrderCreatedEvent event) {
    if (event.total().signum() < 0) {
        throw new IllegalArgumentException("Invalid order total");
    }
}
```

Without proper error handling, the consumer may keep retrying the same bad message.

A common pattern is a dead-letter topic, often called a DLT.

```
orders.created
orders.created.DLT
```

If processing fails repeatedly, the bad message is moved to the DLT so normal processing can continue. Developers can inspect the DLT later.

Conceptually:

```
Message received
|
Process message
|
```

```
Success? yes ---> commit offset
|
no
|
Retry a few times
|
Still failing? ---> send to dead-letter topic
```

Testing Kafka with Embedded Kafka

For tests, Spring Kafka supports embedded Kafka. This lets you test producer and consumer behavior without needing a real Kafka server running separately.

A typical test checks:

```
Given an OrderCreatedEvent
When the producer publishes it
Then the expected message appears on the Kafka topic
```

Another useful test checks:

```
Given a message on orders.created
When the consumer receives it
Then the consumer processes it correctly
```

You are not just unit testing a method. You are testing message flow.

When Kafka Is a Good Fit

Kafka is a good fit when the business event matters beyond one service.

Good examples:

```
OrderCreated
PaymentCompleted
UserRegistered
InventoryChanged
ShipmentDispatched
```

These are events. Other services may react to them independently.

Kafka is usually not the best fit when a service needs an immediate answer from another service. In that case, REST, gRPC, or another synchronous call may be simpler.

Practice Exercises

Exercise 1: Basic Producer

Create a Spring Boot service that publishes an `OrderCreatedEvent` to a Kafka topic.

Practice:

```
OrderCreatedEvent(orderId, customerId, totalAmount)
```

Goal:

```
POST /orders
-> simulate order creation
-> publish event to Kafka topic orders.created
```

Focus on `KafkaTemplate` .

Exercise 2: Basic Consumer

Create a consumer that listens to `orders.created` .

Goal:

```
When an order event is published,
Inventory Service receives it and logs:
"Reserving inventory for order 101"
```

Focus on `@KafkaListener` .

Exercise 3: Use JSON Messages

Send and receive a structured Java object instead of plain strings.

Example event:

```
public record PaymentCompletedEvent(
    Long paymentId,
    Long orderId,
    BigDecimal amount,
    String status
) {}
```

Goal: configure JSON serialization and deserialization correctly.

Exercise 4: Consumer Groups

Run two instances of the same consumer with the same `groupId` .

Observe:

```
Only one instance processes each message.
```

Then run consumers with different `groupId` values.

Observe:

```
Each group receives its own copy of the message.
```

This is one of the most important Kafka concepts to actually see in action.

Exercise 5: Multiple Services Reacting to One Event

Publish one event:

```
orders.created
```

Create three consumers:

```
inventory-service  
email-service  
analytics-service
```

Each should use a different consumer group.

Goal: one published event triggers multiple independent reactions.

Exercise 6: Invalid Message Error Handling

Make the consumer throw an exception when the order total is invalid.

Example:

```
if (event.totalAmount().compareTo(BigDecimal.ZERO) <= 0) {  
    throw new IllegalArgumentException("Invalid order total");  
}
```

Goal: understand what happens when message processing fails.

Exercise 7: Dead-Letter Topic

Configure failed messages to go to:

```
orders.created.DLT
```

Goal:

```
Valid message -> processed normally  
Invalid message -> retried, then sent to dead-letter topic
```

This is a practical production skill.

Exercise 8: Embedded Kafka Test

Write an integration test that starts embedded Kafka.

Test:

```
Given an OrderCreatedEvent  
When producer publishes it
```

```
Then consumer receives it
```

Focus on testing Kafka behavior without depending on a manually running Kafka broker.

Exercise 9: Event Key Ordering

Publish multiple events for the same order using the same key.

Example:

```
order-101: CREATED
order-101: PAID
order-101: SHIPPED
```

Goal: understand why event keys matter for ordering.

Exercise 10: Mini Project

Build a small event-driven order flow:

```
Order API
-> publishes OrderCreatedEvent

Inventory Consumer
-> reserves stock
-> publishes InventoryReservedEvent

Payment Consumer
-> listens for InventoryReservedEvent
-> publishes PaymentCompletedEvent

Notification Consumer
-> listens for PaymentCompletedEvent
-> sends confirmation log
```

No real database is required at first. Logs are enough.

Hands-On Lab: Kafka Producer and Consumer in Spring Boot

Lab Goal

Create a Spring Boot app that publishes an `OrderCreatedEvent` to Kafka and consumes it using `@KafkaListener`.

What You Will Build

```
POST /orders
|
v
OrderController
|
```



```
      v
KafkaTemplate sends OrderCreatedEvent
      |
      v
Kafka topic: orders.created
      |
      v
@KafkaListener receives event
      |
      v
Logs: "Inventory reserved for order..."
```

Step 1: Create Spring Boot Project

Use these dependencies:

```
Spring Web
Spring for Apache Kafka
Lombok optional
```

If using Maven, your main Kafka dependency is:

```
<dependency>
  <groupId>org.springframework.kafka</groupId>
  <artifactId>spring-kafka</artifactId>
</dependency>
```

Step 2: Run Kafka Locally

Use Docker Compose:

```
services:
  kafka:
    image: apache/kafka:latest
    container_name: kafka
    ports:
      - "9092:9092"
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@localhost:9093
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
```

Start Kafka:

```
docker compose up -d
```

Step 3: Configure Spring Boot

In `application.yml` :

```
spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
    consumer:
      group-id: inventory-service
      auto-offset-reset: earliest
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.JsonDeserializer
      properties:
        spring.json.trusted.packages: "*"

```

Step 4: Create Event Class

```
package com.example.kafkaapp.events;

import java.math.BigDecimal;

public record OrderCreatedEvent(
    Long orderId,
    Long customerId,
    BigDecimal totalAmount
) {
}

```

Step 5: Create Producer

```
package com.example.kafkaapp.producer;

import com.example.kafkaapp.events.OrderCreatedEvent;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.stereotype.Service;

@Service
public class OrderEventProducer {

    private final KafkaTemplate<String, OrderCreatedEvent> kafkaTemplate;

    public OrderEventProducer(KafkaTemplate<String, OrderCreatedEvent> kafkaTemplate) {
        this.kafkaTemplate = kafkaTemplate;
    }
}

```

```

    }

    public void sendOrderCreated(OrderCreatedEvent event) {
        kafkaTemplate.send("orders.created", event.orderId().toString(), event);
    }
}

```

Step 6: Create REST Controller

```

package com.example.kafkaapp.controller;

import com.example.kafkaapp.events.OrderCreatedEvent;
import com.example.kafkaapp.producer.OrderEventProducer;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/orders")
public class OrderController {

    private final OrderEventProducer producer;

    public OrderController(OrderEventProducer producer) {
        this.producer = producer;
    }

    @PostMapping
    public String createOrder(@RequestBody OrderCreatedEvent event) {
        producer.sendOrderCreated(event);
        return "Order event published: " + event.orderId();
    }
}

```

Step 7: Create Consumer

```

package com.example.kafkaapp.consumer;

import com.example.kafkaapp.events.OrderCreatedEvent;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.stereotype.Service;

@Service
public class InventoryConsumer {

    @KafkaListener(topics = "orders.created", groupId = "inventory-service")
    public void consume(OrderCreatedEvent event) {
        System.out.println("Inventory reserved for order: " + event.orderId());
    }
}

```

```
        System.out.println("Customer: " + event.customerId());
        System.out.println("Total: " + event.totalAmount());
    }
}
```

Step 8: Test with Postman or curl

Start the Spring Boot app, then send:

```
curl -X POST http://localhost:8080/orders \
-H "Content-Type: application/json" \
-d "{\"orderId\":\"101\",\"customerId\":\"55\",\"totalAmount\":\"249.99\"}"
```

Expected API response:

```
Order event published: 101
```

Expected console output:

```
Inventory reserved for order: 101
Customer: 55
Total: 249.99
```

Step 9: Practice Challenge

Add a second consumer:

```
EmailConsumer
```

It should listen to the same topic but use a different group ID:

```
@KafkaListener(topics = "orders.created", groupId = "email-service")
```

Expected result:

```
InventoryConsumer receives the event
EmailConsumer also receives the event
```

Step 10: Lab Completion Checklist

You are done when:

```
Kafka runs locally
Spring Boot starts successfully
POST /orders publishes an event
KafkaTemplate sends JSON to orders.created
@KafkaListener receives the event
```

Console logs show the consumed order
A second consumer can also react to the same event

Review Questions

1. Why should `OrderService` publish `OrderCreatedEvent` instead of directly calling Email, Inventory, and Analytics services?
2. What is the difference between two consumers in the same group and two consumers in different groups?
3. Why is a dead-letter topic useful?
4. What does `KafkaTemplate` do in Spring Boot?
5. Why do Kafka messages need serializers?

Recommended Practice Order

1. Producer with `KafkaTemplate`
2. Consumer with `@KafkaListener`
3. JSON event objects
4. Consumer groups
5. Multiple consumers
6. Error handling
7. Dead-letter topic
8. Embedded Kafka tests
9. Event keys and ordering
10. Mini event-driven order system